`stride_view`

Document #:	P1899R1
Date:	2021-11-07
Project:	Programming Language C++
Audience:	Ranges Study Group, Library Evolution Working Group

Reply-to:	Christopher Di Bella <cjdb.ns@gmail.com>

Contents

§ 1 Abstract

The ability to use algorithms over an evenly-spaced subset of a range has been missed in the STL for a quarter of a century. Given that there's no way to compose a strided range adaptor in C++20, this should be adopted for C++23.

§ 2 Revision history

§ 2.1 R1

— PDF -> HTML.

— Adds a section discussing the design.

— Adds feature-test macro.

— Cleans up some stuff that was ported in from the implementation by mistake.

— Adds `iterator_concept`, and corrects `iterator_category` so it can't be contiguous.

— Fixes calls to *`compute-distace`* so they pass in size of underlying range instead of themselves.

— Adds precondition to ensure stride is positive.

— Makes multi-arg constructors non-explicit.

§ 2.2 R0

Initial revision.

§ 3 Motivation

The ability to use algorithms over an evenly-spaced subset of a range has been missed in the STL for a quarter of a century. This is, in part, due to the complexity required to use an iterator that can safely describe such a range. It also means that the following examples cannot be transformed from raw loops into algorithms, due to a lacking iterator.

```
namespace stdr = std::ranges;
namespace stdv = std::views;

for (auto i = 0; i < ssize(v); i += 2) {
  v[i] = 42; // fill
}

for (auto i = 0; i < ssize(v); i += 3) {
  v[i] = f(); // transform
}

for (auto i = 0; i < ssize(v); i += 3) {
  for (auto j = i; j < ssize(v); i += 3) {
    if (v[j] < v[i]) {
      stdr::swap(v[i], v[j]); // selection sort, but hopefully the idea is
        conveyed
    }
  }
}
```

Boost.Range 2.0 introduced a range adaptor called `strided`, and range-v3's equivalent is `stride_view`, both of which make striding far easier than when using iterators:

```
stdr::fill(v | stdv::stride(2), 42);

auto strided_v = v | stdv::stride(3);
stdr::transform(strided_v, stdr::begin(strided_v) f);

stdr::stable_sort(strided_v); // order restored!
```

Given that there's no way to compose a strided range adaptor in C++20, this should be one of the earliest range adaptors put into C++23.

§ 3.1 Risk of not having `stride_view`

Although it isn't possible to compose `stride_view` in C++20, someone inexperienced with the ranges design space might mistake `filter_view` as a suitable way to "compose" `stride_view`:

```
auto bad_stride = [](auto const step) {
  return views::filter([n = 0, step](auto&&) mutable {
    return n++ % step == 0;
  });
};
```

This implementation is broken for two reasons:

1. `filter_view` expects a `predicate` as its input, but the lambda we have provided does not model `predicate` (a call to `invoke` on a `predicate` mustn't modify the function object, yet we clearly are).
2. The lambda provided doesn't account for moving backward, so despite *satisfying* `bidirectional_iterator`, it does not model the concept, thus rendering any program containing it ill-formed, with no diagnostic being required.

For these reasons, the author regrets not proposing this in the C++20 design space.

§ 4 Implementation experience

Both Boost.Range 2.0 and range-v3 are popular ranges libraries that support a striding range adaptor. The proposed wording has mostly been implemented in cmcstl2 and in a CppCon main session.

§ 5 Design notes

§ 5.1 Preconditions

Boost.Range 2.0's `strided` has a precondition that `0 <= n`, but this isn't strong enough: we need `n` to be *positive*.

The stride needs to be positive since a negative stride doesn't really make sense, and a semantic requirement of `std::weakly_incrementable` (23.3.4.4 [iterator.concept.winc])

is that incrementing actually moves the iterator to the next element: this means a zero-stride isn't allowed either.

LEWG unanimously agreed that this was the correct decision in Prague.

§ 5.2 Complex iteration model

A simple implementation of `stride_view` would be similar to what's in Boost.Range 2.0: a single-pass range adaptor. With some effort, we can go all the way to a random-access range adaptor, which is what this section mainly covers.

A naive random-access range adaptor would be implemented by simply moving the iterator forward or backward by `n` positions (where `n` is the stride length). While this produce a correct iterator when moving forward, its `operator--` will be incorrect whenever `n` doesn't evenly divide the underlying range's length. For example:

```
auto x = std::vector{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11};

// prints 0 3 6 9
stdr::copy(stdv::stride(x, 3), std::ostream_iterator<int>(std::cout, "
        "));

// prints 9 6 3 0
stdr::copy(stdv::stride(x, 3) | stdv::reverse, std::ostream_iterator<int>
        (std::cout, " "));

auto y = std::vector{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10};

// prints 0 3 6 9
stdr::copy(stdv::stride(y, 3), std::ostream_iterator<int>(std::cout, "
        "));

// prints 8 5 2: not the same range in reverse!?
stdr::copy(stdv::stride(y, 3) | stdv::reverse, std::ostream_iterator<int>
        (std::cout, " "));
```

The problem here is that the range has lost all information by the time we've reached the end of the sequence. In order to correctly iterate backwards, we need to cache our step value and do some fancy computation:

```
// `stride_` is the number of elements we're supposed to skip over
// `n` is the number of strides we take
if (moving_forward) {
  auto remaining = ranges::advance(current_, n * stride_,
        ranges::end(underlying_range_));
  step_ = stride_ - remaining;
  return *this;
}
```

When moving forward, we update our `step_` cache. In most cases, this will be zero, since we'll usually be taking a stride of length `stride_`. The only case where this `step_` is a

number is when there are fewer elements left to skip than our stride length. This is important for preserving the range when iterating backward.

```
if (moving_backward) {
  auto stride = step_ == 0 ? n * stride_
                           : (n + 1) * stride_ - step_;
  step_ = ranges::advance(current_, stride,
        ranges::begin(underlying_range_));
}
```

When we move backward, we consider the value of `step_`. If it's zero, then we skip over `n * stride_` elements in the same way as moving forward. If `step_` is nonzero, then we need to skip over fewer elements in its first step, to make up the distance. In the example above, to print out `9 6 3 0`, `y`'s iterator needs to pretend there's one extra value between the end of the underlying range and `10` so that the first value of our `stdv::stride(y, 3) | stdv::reverse` is `9`.

§ 6 Proposed wording

§ 6.1 Range adaptors [range.adaptors]

§ 6.1.1 Header <ranges> synopsis [ranges.syn]

Add the following to 24.2 [ranges.syn]:

```
// [...]
namespace std::ranges {
  // [...]

  // [range.stride]
  template<input_range R>
    requires view<R>
  class stride_view;

  template<class R>
    inline constexpr bool enable_borrowed_range<stride_view<R>> =
      forward_range<R> && enable_borrowed_range<R>;

  namespace views { inline constexpr unspecified stride = unspecified; }

  // [...]
}
```

§ 6.1.2 Stride view [range.stride]

Add the contents of this subsection as a subclause to 24.7 [range.adaptors].

§ 6.1.2.1 Overview [range.stride.overview]

1 `stride_view` presents a view of an underlying sequence, advancing over `n` elements at a time, as opposed to the usual single-step succession.

2 The name `views::stride` denotes a range adaptor object 24.7.2 [range.adaptor.object]. Given subexpressions `E` and `N`, the expression `views::stride(E, N)` is expression-equivalent to `stride_view(E, N)`.

3 [*Example*:

```
auto input = views::iota(0, 12) | views::stride(3);
ranges::copy(input, ostream_iterator<int>(cout, " ")); // prints 0 3 6 9
ranges::copy(input | views::reverse, ostream_iterator<int>(cout, " ")); //
        prints 9 6 3 0
```

— *end example*]

§ **6.1.2.2 Class `stride_view` [range.stride.view]**

```
namespace std::ranges {
  template<input_range R>
    requires view<R>
  class stride_view : public view_interface<stride_view<R>> {
    template<bool Const> class iterator; // exposition only
  public:
    stride_view() = default;
    constexpr stride_view(R base, range_difference_t<R> stride);

    constexpr R base() const& requires copy_constructible<R> { return
        base_; }
    constexpr R base() && { return std::move(base_); }

    constexpr range_difference_t<R> stride() const noexcept;

    constexpr iterator<false> begin() requires (!simple-view<R>);
    constexpr iterator<true> begin() const requires range<const R>;

    constexpr auto end() requires (!simple-view<R>) {
      if constexpr (!bidirectional_range<R> || (sized_range<R> &&
        common_range<R>)) {
        return iterator<false>(*this, ranges::end(base_),
        ranges::distance(base_) % stride_);
      }
      else {
        return default_sentinel;
      }
    }

    constexpr auto end() const requires range<const R> {
      if constexpr (!bidirectional_range<R> || (sized_range<R> &&
        common_range<R>)) {
        return iterator<true>(*this, ranges::end(base_),
        ranges::distance(base_) % stride_);
      }
      else {
        return default_sentinel;
```

```
      }
    }

    constexpr auto size() requires (sized_range<R> && !simple-view<R>) {
      return compute-distance(ranges::size(base_));
    }

    constexpr auto size() const requires sized_range<const R> {
      return compute-distance(ranges::size(base_));
    }
  private:
    R base_;                            // exposition only
    range_difference_t<R> stride_ = 1; // exposition only

    template<class I>
    constexpr I compute-distance(I distance) const { // exposition only
      const auto quotient = distance / static_cast<I>(stride_);
      const auto remainder = distance % static_cast<I>(stride_);
      return quotient + static_cast<I>(remainder > 0);
    }
  };

  template<class R>
    stride_view(R&&, range_difference_t<R>) ->
        stride_view<views::all_t<R>>;
}
```

```
constexpr stride_view(R base, range_difference_t<R> stride);
```

1 *Preconditions*: `stride > 0`.

2 *Effects*: Initializes `base_` with `base` and `stride_` with `stride`.

```
constexpr R base() const;
```

3 *Effects*: Equivalent to `return base_;`

```
constexpr range_difference_t<R> stride() const;
```

4 *Effects*: Equivalent to `return stride_;`

```
constexpr iterator<false> begin() requires (!simple-view<R>);
```

5 *Effects*: Equivalent to `return iterator<false>(*this);`

```
constexpr iterator<true> begin() requires (!simple-view<R>);
```

6 *Effects*: Equivalent to `return iterator<true>(*this);`

§ ### 6.1.2.3 Class `stride_view::iterator` [range.stride.iterator]

```
namespace std::ranges {
  template<input_range R>
    requires view<R>
  template<bool Const>
```

```
class stride_view<R>::iterator {
  using Parent = conditional_t<Const, const stride_view, stride_view>;
      // exposition only
  using Base = conditional_t<Const, const R, R>;
      // exposition only

  friend iterator<!Const>;

  Parent* parent_;                // exposition only
  iterator_t<Base> current_;      // exposition only
  range_difference_t<Base> step_; // exposition only
public:
  using difference_type = range_difference_t<Base>;
  using value_type = range_value_t<Base>;
  using iterator_concept = see below;
  using iterator_category = see below; // not always present

  iterator() = default;

  constexpr explicit iterator(Parent& parent);
  constexpr iterator(Parent& parent, iterator_t<Base> end,
      difference_type step);
  constexpr explicit iterator(iterator<!Const> other)
    requires Const && convertible_to<iterator_t<R>, iterator_t<Base>>;

  constexpr iterator_t<Base> base() const;

  constexpr decltype(auto) operator*() const { return *current_; }

  constexpr iterator& operator++();

  constexpr void operator++(int);
  constexpr iterator operator++(int) requires forward_range<Base>;

  constexpr iterator& operator--() requires bidirectional_range<Base>;
  constexpr iterator operator--(int) requires bidirectional_range<Base>;

  constexpr iterator& operator+=(difference_type n) requires
      random_access_range<Base>;
  constexpr iterator& operator-=(difference_type n) requires
      random_access_range<Base>;

  constexpr decltype(auto) operator[](difference_type n) const
    requires random_access_range<Base>
  { return *(*this + n); }

  constexpr iterator& operator+(const iterator& x, difference_type n)
    requires random_access_range<Base>;

  constexpr iterator& operator+(difference_type n, const iterator& x)
    requires random_access_range<Base>;

  constexpr iterator& operator-(const iterator& x, difference_type n)
    requires random_access_range<Base>;
```

```
  constexpr difference_type operator-(const iterator& x, const iterator&
      y)
    requires random_access_range<Base>;

  constexpr friend bool operator==(const iterator& x, default_sentinel);

  constexpr friend bool operator==(const iterator& x, const iterator& y)
    requires equality_comparable<iterator_t<Base>>;

  constexpr friend bool operator<(const iterator& x, const iterator& y)
    requires random_access_range<Base>;

  constexpr friend bool operator>(const iterator& x, const iterator& y)
    requires random_access_range<Base>;

  constexpr friend bool operator<=(const iterator& x, const iterator& y)
    requires random_access_range<Base>;

  constexpr friend bool operator>=(const iterator& x, const iterator& y)
    requires random_access_range<Base>;

  constexpr friend compare_three_way_result_t<iterator_t<Base>>
    operator<=>(const iterator& x, const iterator& y)
      requires random_access_range<Base> &&
      three_way_comparable<iterator_t<Base>>;

  constexpr friend range_rvalue_reference_t<R> iter_move(const iterator&
      i)
    noexcept(noexcept(ranges::iter_move(i.current_)));

  constexpr friend void iter_swap(const iterator& x, const iterator& y)
    noexcept(noexcept(ranges::iter_swap(x.current_, y.current_)))
    requires indirectly_swappable<iterator_t<R>>;
private:
  constexpr iterator& advance(difference_type n) { // exposition only
    if constexpr (!bidirectional_range<Parent>) {
      ranges::advance(current_, n * parent_->stride_,
      ranges::end(parent_->base_));
      return *this;
    }
    else {
      if (n > 0) {
        auto remaining = ranges::advance(current_, n * parent_->stride_,
       ranges::end(parent_->base_));
        step_ = parent_->stride_ - remaining;
        return *this;
      }

      if (n < 0) {
        auto stride = step_ == 0 ? n * parent_->stride_
                                 : (n + 1) * parent_->stride_ - step_;
        ranges::advance(current_, stride);
        stride_ = 0;
        return *this;
      }
```

```
        return *this;
      }
    }
  };
}
```

1 *iterator*::iterator_concept is defined as follows:

(1.1) — If R models random_access_range, then iterator_concept denotes random_access_iterator_tag.

(1.2) — Otherwise, if R models bidirectional_range, then iterator_concept denotes bidirectional_iterator_tag.

(1.3) — Otherwise, if R models forward_range, then iterator_concept denotes forward_iterator_tag.

(1.3) — Otherwise, iterator_concept denotes input_iterator_tag.

2 The member *typedef-name* iterator_category is defined if and only if R models forward_range. In that case, *iterator*::iterator_category is defined as follows:

(2.1) — Let C denote the type iterator_traits<iterator_t<R>>::iterator_category.

(2.2) — If C models derived_from<random_access_iterator_tag>, then iterator_category denotes random_access_iterator_tag.

(2.3) — Otherwise, iterator_category denotes C.

```
constexpr explicit iterator(Parent& parent);
```

3 *Effects*: Initializes parent_ with addressof(parent) and current_ with ranges::begin(parent).

```
constexpr iterator(Parent& parent, iterator_t<Base> end, difference_type
        step);
```

4 *Effects*: Initializes parent_ with addressof(parent) and current_ with std::move(end), and step_ with step.

```
constexpr explicit iterator(iterator<!Const> other)
  requires Const && convertible_to<iterator_t<R>, iterator_t<Base>>;
```

5 *Effects*: Initializes parent_ with other.parent_ and current_ with std::move(other.current_), and step_ with other.step_.

```
constexpr iterator& operator++();
```

6 *Effects*: Equivalent to: return advance(1);

```
constexpr void operator++(int);
```

7 *Effects*: Equivalent to: advance(1);

```
constexpr iterator operator++(int) requires forward_range<Base>;
```

8 *Effects*: Equivalent to:

```
auto temp = *this;
++*this;
return temp;
```

```
constexpr iterator& operator--() requires bidirectional_range<Base>;
```

9 *Effects*: Equivalent to: `return advance(-1);`

```
constexpr iterator operator--(int) requires bidirectional_range<Base>;
```

```
auto temp = *this;
--*this;
return temp;
```

```
constexpr iterator& operator+=(difference_type n) requires
        random_access_range<Base>;
```

10 *Effects*: Equivalent to: `return advance(n);`

```
constexpr iterator& operator-=(difference_type n) requires
        random_access_range<Base>;
```

11 *Effects*: Equivalent to: `return advance(-n);`

```
constexpr iterator& operator+(iterator x, difference_type n)
  requires random_access_range<Base>;
```

12 *Effects*: Equivalent to: `return x += n;`

```
constexpr iterator& operator+(difference_type n, iterator x)
  requires random_access_range<Base>;
```

13 *Effects*: Equivalent to: `return x += n;`

```
constexpr iterator& operator-(const iterator& x, difference_type n)
  requires random_access_range<Base>;
```

14 *Effects*: Equivalent to: `return x -= n;`

```
constexpr difference_type operator-(const iterator& x, const iterator& y)
  requires random_access_range<Base>;
```

15 *Effects*: Equivalent to: `return x.parent_->`*`compute-distance`*`(x.current_ - y.current_);`

```
constexpr friend bool operator==(const iterator& x, default_sentinel);
```

16 *Effects*: Equivalent to: `return x.current_ == ranges::end(x.parent_->base_);`

```
constexpr friend bool operator==(const iterator& x, const iterator& y)
      requires equality_comparable<iterator_t<Base>>;
```

16 *Effects*: Equivalent to: `return x.current_ == y.current_;`

```
constexpr friend bool operator<(const iterator& x, const iterator& y)
  requires random_access_range<Base>;
```

17 *Effects*: Equivalent to: `return x.current_ < y.current_;`

```
constexpr friend bool operator>(const iterator& x, const iterator& y)
  requires random_access_range<Base>;
```

18 *Effects*: Equivalent to: `return y < x;`

```
constexpr friend bool operator<=(const iterator& x, const iterator& y)
  requires random_access_range<Base>;
```

19 *Effects*: Equivalent to: `return !(y < x);`

```
constexpr friend bool operator>=(const iterator& x, const iterator& y)
  requires random_access_range<Base>;
```

20 *Effects*: Equivalent to: `return !(x < y);`

```
constexpr friend compare_three_way_result_t<iterator_t<Base>>
  operator<=>(const iterator& x, const iterator& y)
    requires random_access_range<Base> &&
        three_way_comparable<iterator_t<Base>>;
```

21 *Effects*: Equivalent to: `return x.current_ <=> y.current_;`

```
constexpr friend range_rvalue_reference_t<R> iter_move(const iterator& i)
  noexcept(noexcept(ranges::iter_move(i.current_)));
```

22 *Effects*: Equivalent to: `return ranges::iter_move(i);`

```
constexpr friend void iter_swap(const iterator& x, const iterator& x)
  noexcept(noexcept(ranges::iter_swap(x.current_, y.current_)))
  requires indirectly_swappable<iterator_t<R>>;
```

23 *Effects*: Equivalent to: `ranges::iter_swap(x.current_, y.current_);`

§ 6.2 Feature-test macro

Add the following macro definition to 17.3.2 [version.syn], header `<version>` synopsis, with the value selected by the editor to reflect the date of adoption of this paper:

```
#define __cpp_lib_ranges_stride 20XXXXL // also in <ranges>
```

§ 7 Acknowledgements

The author would like to thank Tristan Brindle for providing editorial commentary on P1899, and also those who reviewed material for, or attended the aforementioned CppCon

session or post-conference class, for their input on the design of the proposed `stride_view`.